

Attention IS All You Need

Wednesday, July 29, 2026 7:55 PM

Analogy:

I am at bank

I am planning for a vacation on ganga bank, and for that I have to book a ticket so to get my money I have to visit bank.

Attention -> Try to learn what is your relationship with all the others.

Self-Attention -> Automatically try to focus yourself on all the others one.

Cons: No Training, Proximity doesn't matter, Order has no influence, Shape Independent.

Chapter - 1 : Why was Transformer Invented?

Problem: Suppose we have a sentence

I love machine learning

An RNN reads it like: I --> love --> machine --> learning

One word at a time. Information has to travel through every previous word.

Issues: Slow, Long-term dependencies fade, Difficult to parallelize.

Transformer asks

Why not allow every word to directly communicate with every other word?

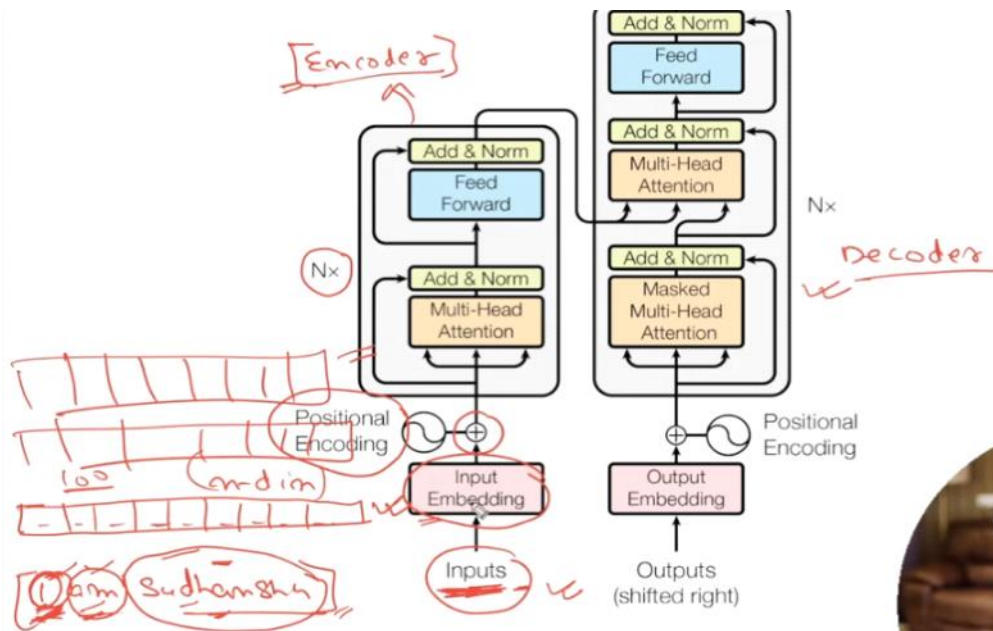
I

↙ ↓ ↘

love machine learning

↘ ↑ ↙

Every word can immediately "look at" every other word. This is the motivation behind attention.



Chapter - 2 (Word Embedding)

Instead of saying: "I" $\rightarrow$ [0.21, -1.4, ...]

Word: "I" $\rightarrow$ Dictionary Lookup $\rightarrow$ Embedding Layer $\rightarrow$ 128-dimensional meaning vector

Imagine every word becoming a coordinate in a semantic space.

King ●



Queen ●

Man ●

Woman ●

Nearby words have similar meanings.



Chapter - 3 (Why Positional Encoding)

Embeddings don't know order. Without position, both contain the same words.

Dog	Bites	Man
Man	Bites	dog

Transformer cannot distinguish them. So we inject position.

Embedding + Position Information --> Final Input

Value: [0.8, -0.2, 0.5, 0.9] (Meaning Vector)

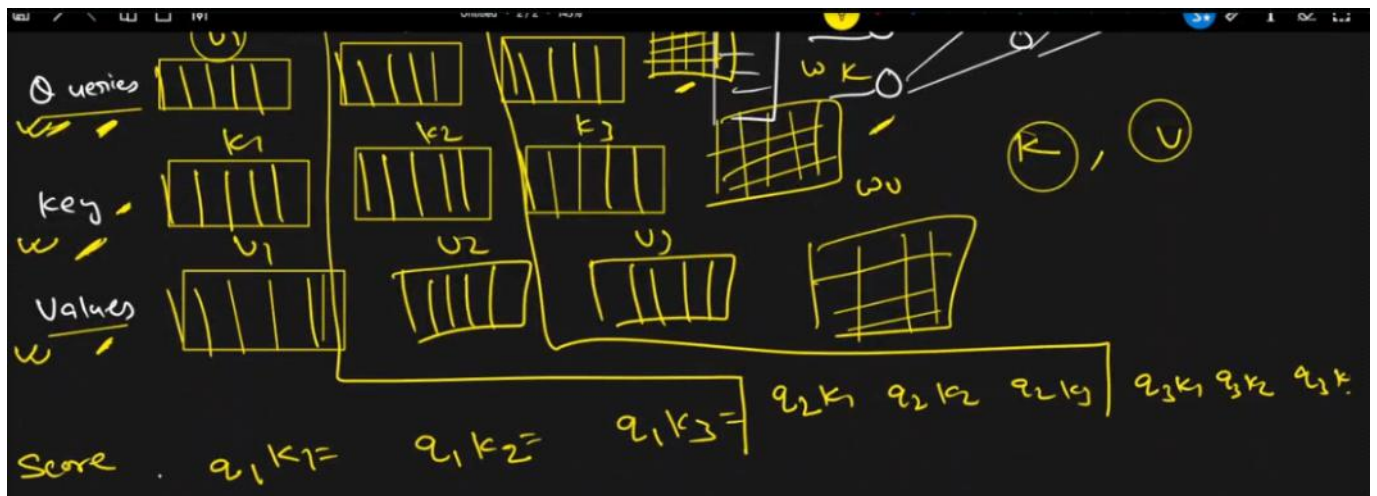
+

Value: [0.1, 0.9, -0.1, 0.2] (Position Vector)

=

Value: [0.9, 0.7, 0.4, 1.1] (Combined Vector)

Chapter - 4 (Query, Key, Value) Google Search Analogy



You type a Query: "Best AI books"

↓

Google compares it against millions of pages,
each page carries a Key: "what is this page about?"

↓

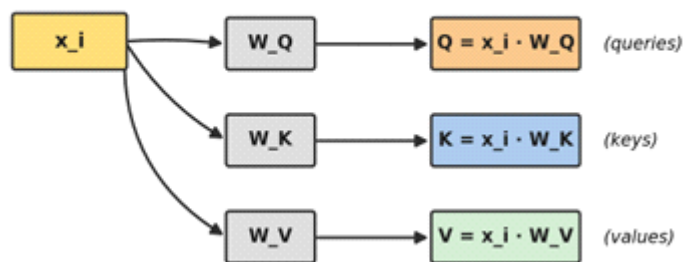
(Query matches Key)

↓

Google returns the Value: the actual page content

Exactly the same happens inside Attention. Every word produces (Q,K,V)

Project each token into Query, Key, Value vectors



Every token's final embedding x_i is multiplied by three learned weight matrices W_Q , W_K , W_V to produce a Query, Key and Value vector for that token.

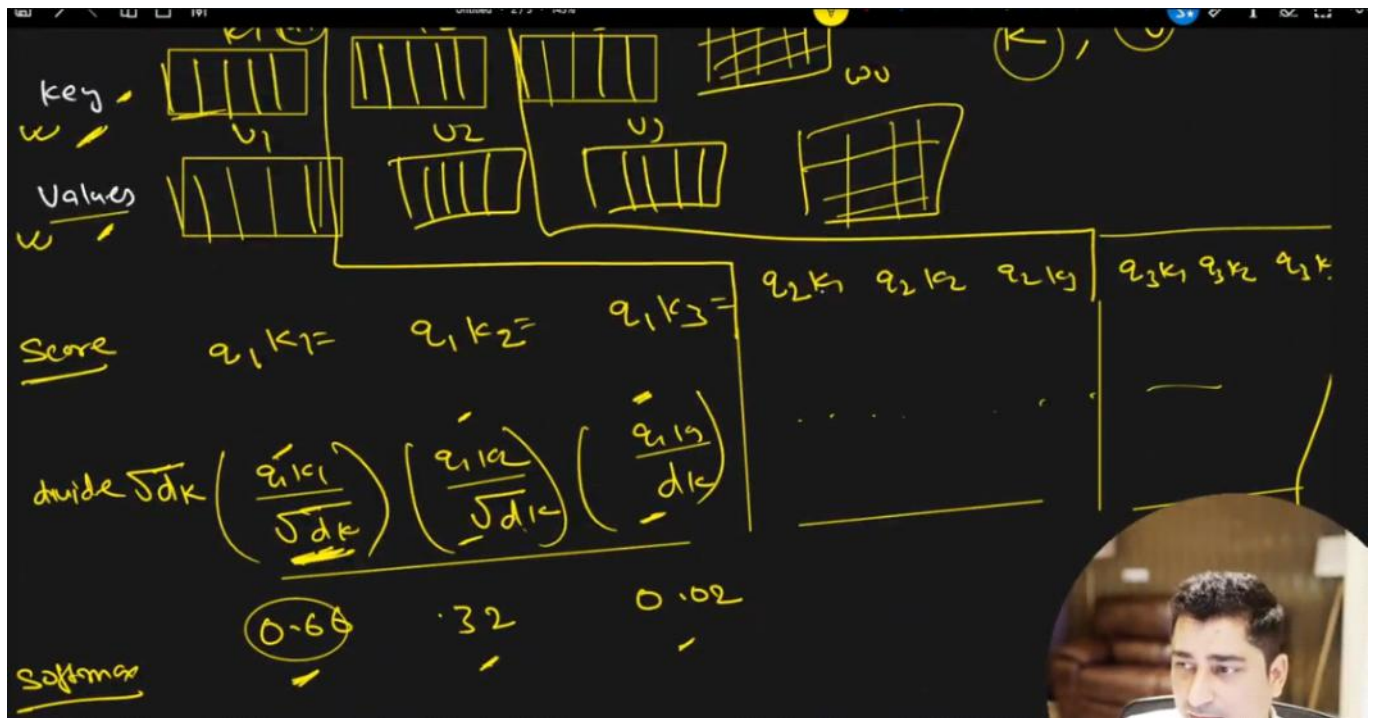
key w v_1 v_2 v_3 w

Values w

Score $q_{1,k1} = q_{1,k2} = q_{1,k3} = q_{2,k1} \ q_{2,k2} \ q_{2,k3} \ | \ q_{3,k1} \ q_{3,k2} \ q_{3,k3}$

divide $\sqrt{d_k} \left(\frac{q_{1,k1}}{\sqrt{d_k}} \right) \left(\frac{q_{1,k2}}{\sqrt{d_k}} \right) \left(\frac{q_{1,k3}}{\sqrt{d_k}} \right)$

Softmax $0.66 \quad 32 \quad 0.02$



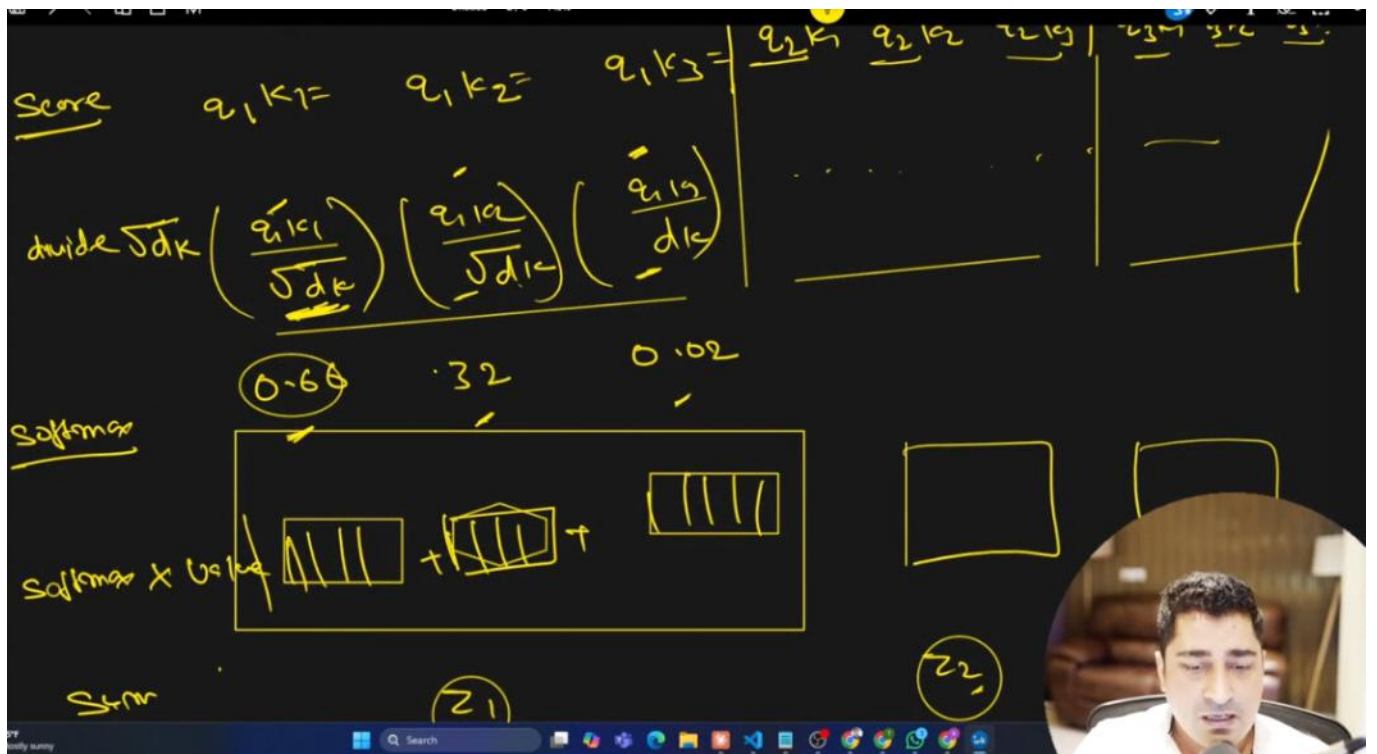
Score $q_{1,k1} = q_{1,k2} = q_{1,k3} = q_{2,k1} \ q_{2,k2} \ q_{2,k3} \ | \ q_{3,k1} \ q_{3,k2} \ q_{3,k3}$

divide $\sqrt{d_k} \left(\frac{q_{1,k1}}{\sqrt{d_k}} \right) \left(\frac{q_{1,k2}}{\sqrt{d_k}} \right) \left(\frac{q_{1,k3}}{\sqrt{d_k}} \right)$

Softmax $0.66 \quad 32 \quad 0.02$

Softmax x Used $\left[\begin{array}{|c|} \hline \text{Diagram} \\ \hline \end{array} \right] + \left[\begin{array}{|c|} \hline \text{Diagram} \\ \hline \end{array} \right] + \left[\begin{array}{|c|} \hline \text{Diagram} \\ \hline \end{array} \right]$

Sum $(21) \quad (22)$

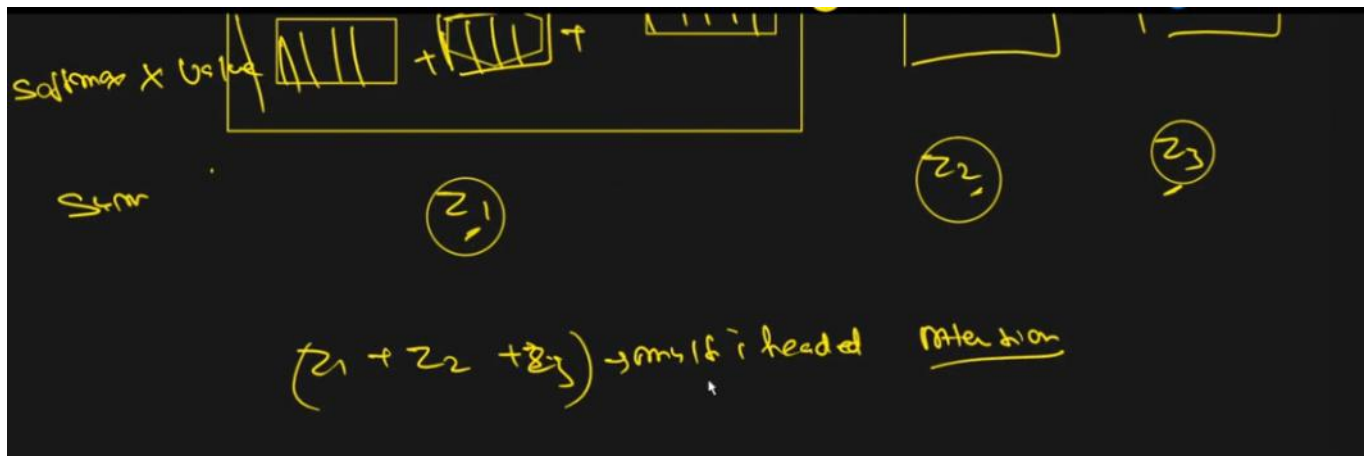


3. Step 2 — Score, Scale, Softmax, Weighted Sum



$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- | | |
|------------------------|--|
| a) Score | $q_1 \cdot k_1, q_1 \cdot k_2, q_1 \cdot k_3$ — the dot product measures how well query 1 ("I") aligns geometrically with each key. Two vectors pointing in a similar direction $\rightarrow$ large dot product $\rightarrow$ high relevance (this is exactly cosine similarity, before normalizing length). |
| b) Scale | Divide every score by $\sqrt{d_k}$ (d_k = key dimension). Without this, dot products grow large as d_k grows, pushing softmax into regions with tiny gradients (near one-hot). Scaling keeps the softmax input variance = 1. |
| c) Softmax | Turns raw scores into a probability distribution that sums to 1, e.g. 0.66, 0.32, 0.02 for (I, am, Sudhanshu) when computing attention for the word "I" — meaning "I" mostly attends to itself, some to "am", almost none to "Sudhanshu". |
| d) Weighted sum | $z_1 = 0.66 \cdot v_1 + 0.32 \cdot v_2 + 0.02 \cdot v_3$. Geometrically, z_1 is a point inside the convex hull of v_1, v_2, v_3 , pulled closest to whichever value vector got the highest weight. |



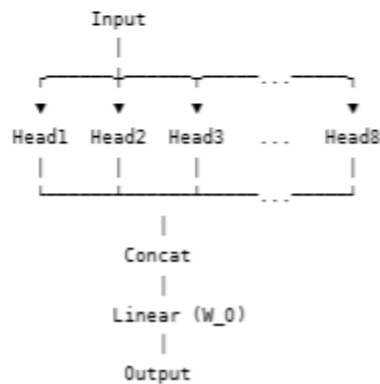
Multi-Head Attention:

Each detective looks for something different. Finally all detectives combine findings.

Imagine 8 detectives investigating the same sentence, each looking for a different kind of clue:

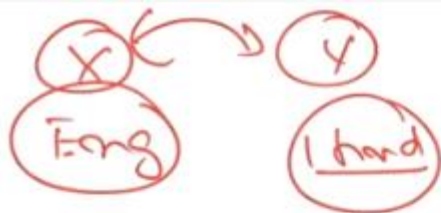
- Head 1 → Grammar
- Head 2 → Meaning
- Head 3 → Subject
- Head 4 → Verb
- Head 5 → Object
- Head 6 → Long-range reference
- Head 7 → Local word order
- Head 8 → General context

Each detective works independently, then they all report back and their findings are combined into one final conclusion.



$(z_1 + z_2 + z_3) \rightarrow \text{multi-headed attention}$

The diagram shows a hand-drawn representation of a multi-headed attention mechanism. On the left, a box contains the expression $z_1 + z_2 + z_3$. An arrow points from this box to a central structure consisting of multiple layers of nodes (circles) connected by lines, representing the attention heads. To the right of this structure is a grid-like box, likely representing the output matrix. The entire diagram is drawn in yellow on a black background.



$$\downarrow \text{loss} = (y - \hat{y})$$

~~1. Self-attention~~ ~~Self-attention~~

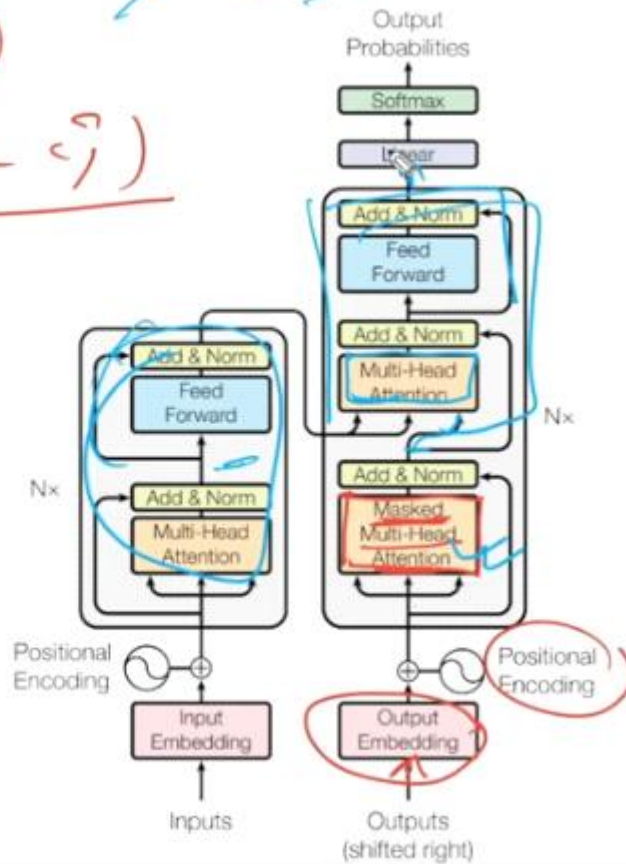


Figure 1: The Transformer - model architecture.

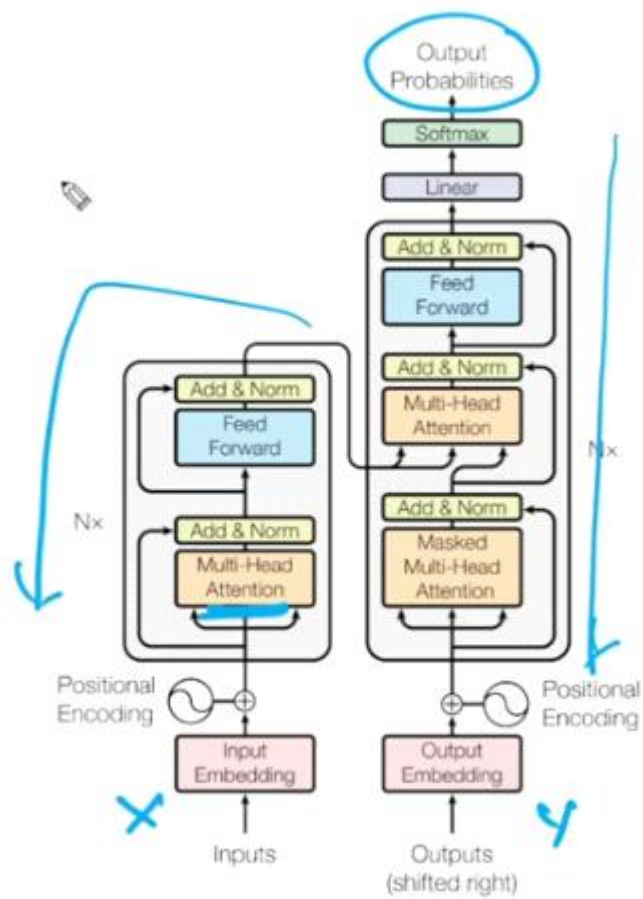


Figure 1: The Transformer - model architecture.